ASE 230 - Project 1

Luyiza Kamelchuk

Project Structure

```
project1/
├── code/
│   ├── schema.sql
│   ├── db.php
│   ├── auth.php
│   ├── index.php
│   ├── test\_curl.sh
│   └── test\_client.html
├── presentation/
│   ├── api\_tutorial.md / .pdf
│   ├── nginx\_deployment.md / .pdf
│   └── nginx\_screenshot.png
├── plan/
│   └── hw2\_plan.md
│   └── hw2\_plan(updated).md
├── README.md
└── project1\_rubric.md
```

#	Endpoint	Description	Bearer
1	GET /students	Get all students	
2	GET /courses	Get all courses	
3	POST /students	Create a new student	
4	POST /courses	Create a new course	
5	GET /students/{id}	Get student by ID	
6	GET /courses/{id}	Get course by ID	
7	PUT /students	Update a student	
8	PUT /courses	Update a course	
9	DELETE /students/{id}	Remove a student by ID	*
10	DELETE /courses/{id}	Remove a course by ID	*

Databases

Table: `students` — id, name, email, major, year Table: `courses` — id, title, department, credits

See `code/schema.sql`

In the next few slides we'll go over how to create and test the Project 1 Database

---

Step 1: Create the database

From the **`project1/`** directory, create the project database and import the initial schema:

```
mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS project1;"
mysql -u root -p project1 < code/schema.sql
```

> [!NOTE] `code/schema.sql` creates the tables and inserts samples (John, Jane, and two courses). It is intended for **initial setup**; running it repeatedly without dropping the database will result in duplicate records. If you ever want to completely wipe and re-initialize a clean database, run:
>
> ```
> mysql -u root -p -e "DROP DATABASE IF EXISTS project1; CREATE DATABASE
> project1;"
> mysql -u root -p project1 < code/schema.sql
> ```

Step 2: Start the PHP Server

`db.php` connects to MySQL using the environment variables `DB\_USER` (defaults to `root`) and `DB\_PASSWORD` (defaults to ase230).

From the **`project1/`** directory:

```
cd code

# If your MySQL root account requires a password (recommended):
DB\_PASSWORD='ase230' php -S localhost:8080
```

The API is now running at `http://localhost:8080/`. Keep this terminal open while running tests.

Step 3: Run the Tests

In a second terminal window, from the `project1/` directory:

*** Automated cURL Tests:**

```
bash code/test\_curl.sh
```

*** Browser Test Client:**

Open the following address in your browser:

```
http://localhost:8080/test\_client.html
```

> [!WARNING]

> **Never open `test\_client.html` directly from your operating system file manager (`file:///...`). Browser security blocks API requests from local file URLs. Always access it through `http://localhost:8080/test\_client.html` while the server is running.**

Curl Test Responses

Code	Meaning
200	OK
201	Created (POST)
400	Missing or invalid input
401	Missing or invalid Bearer token
404	Resource or record not found
405	Method not allowed on this path
409	A student with that email already exists
500	Database error

We should see a '200' code for every test (unless testing error codes)

GET /students - get all students

Request:

```
curl http://localhost:8080/students
```

GET /courses - get all courses

Request:

```
curl http://localhost:8080/students
```

POST /students - create new student

```
curl -X POST http://localhost:8080/students \
  -H "Content-Type: application/json" \
  -d '{"name":"Test
Student","email":"test@example.com","major":"Cybersecurity","year":1}'
```

POST /courses - create new couse

```
curl -X POST http://localhost:8080/courses \
  -H "Content-Type: application/json" \
   -d '{"title":"Test Course","department":"Cybersecurity","credits":3}'
```

GET /students/{id} - get student by ID

```
bash curl http://localhost:8080/students/1
```

GET /courses/{id} - get course by ID

```
bash curl http://localhost:8080/courses/1
```

PUT /students — update a student

```
curl -X PUT http://localhost:8080/students \
  -H "Content-Type: application/json" \
  -d '{"id":1,"name":"John
Examples","email":"johnexample@nku.edu","major":"Computer Science","year":3}'
```

PUT /courses — update a course

```
curl -X PUT http://localhost:8080/courses \
  -H "Content-Type: application/json" \
  -d '{"id":1,"title":"Updated Course
Name","department":"Cybersecurity","credits":4}'
```

DELETE /students/{id} - delete student by id (Bearer Token)

Needs a Bearer token in the Authorization header.

```
curl -X DELETE http://localhost:8080/students/1 \
  -H "Authorization: Bearer ase230-token"
```

*This will return an error without a token

DELETE /courses/{id} - delete course by id (Bearer Token)

Needs a Bearer token in the Authorization header.

```
curl -X DELETE http://localhost:8080/courses/1 \
  -H "Authorization: Bearer ase230-token"
```

*This will return an error without a token

Step 4: Stop the Server

When finished testing, return to the first terminal where the PHP server is running and press `Ctrl+C`.

You have successfully complete the project!

Additional References used throughout the project

https://www.zend.com/blog/curl-in-php https://onlinephp.io/ https://medium.com/@hammadbawara/inserting-sample-data-into-a-database-using-sql-queries-a9501c7d9443 https://www.datacamp.com/tutorial/sql-remove-duplicates https://www.w3schools.com/sql/sql_insert.asp https://softwareengineering.stackexchange.com/questions/249484/how-to-update-a-row-in-database-without-id https://stackoverflow.com/questions/77404866/how-to-request-a-bearer-token-directly-from-sql-server-using-a-certificate https://stackoverflow.com/questions/71364092/how-to-add-bearer-token-to-a-request-in-asp-net-core https://better-auth.com/docs/plugins/bearer

---

https://www.fastly.com/blog/anatomy-of-a-curl-how-to-use-curl-to-test-an-origin-servers-response https://crawlbase.com/blog/how-to-send-get-requests-with-curl/ https://octopus.com/devops/software-deployments/php-with-nginx/ https://dev.to/anitaalicloud/building-and-deploying-a-simple-rest-api-on-a-linux-vps-with-nginx-4l10 https://nginx.org/en/docs/beginners_guide.html